

Shared PPO Update Core (optimize_batches)

Source: `crl/ppo/trainer.py` (PPOTrainer.optimize_batches)

Used by: LocalTrainer, GlobalTrainer (Formulation A), StoredExpertTrainer (Formulation B), and the joint-training experiment.

Math: standard clipped PPO; the per-stream actor_coeff carries all the continual-learning weighting (eqs 22-29, 46). This routine itself is task-agnostic -- "nothing continual lives here."

Idea

One PPO update = `ppo\_epochs` passes of minibatch SGD over one or more collected rollout STREAMS. Each stream *i* contributes:

```
actor   : actor_coeffs[i] * clipped_surrogate_i
critic  : vf_coef * value_MSE_i                (standard, always coeff 1)
entropy: - ent_coef * entropy_i                (bonus)
```

summed into a single scalar loss, then one Adam step under a global grad-norm clip. The ONLY thing that changes between local / global / stored-expert is the list `actor\_coeffs`; the optimization is shared verbatim.

Key stabilizer: actor coefficients are RENORMALIZED by their sum before use, so the actor-loss scale stays $O(1)$ no matter how large the dual coefficient $\mu^2 \cdot \text{shortfall}$ grows. Without it, a large μ makes the actor gradient dominate the SHARED grad-norm clip and starves the shared critic's value-loss gradient, breaking GAE advantages (memory: PPO shared-critic starvation).

Inputs

```
policy      : shared-trunk multi-head actor+critic
optimizer   : Adam over policy's trainable params (lr, eps=1e-5)
streams     : list of RolloutBatch (obs, actions, logprobs, advantages,
                      returns, task_id) -- one per task, each already GAE-processed
actor_coeffs: per-stream actor weight
              local      : [1.0]
              global (A) : [ $\omega_1.. \omega_{k-1}$ ,  $\mu^2 \cdot \text{shortfall}$ ]
              stored-expert(B): [ $\omega_i * 2 * \text{gap}_i$  for all seen tasks]
bc (optional): (local_policy, bc_coef) -> add KL( $\pi_{\text{local}} || \pi_{\text{global}}$ ) on the
              CURRENT (last) stream (Experiment 2 behavioral cloning)
cfg: num_minibatches, ppo_epochs, clip_ratio, vf_coef, ent_coef,
     normalize_advantage, max_grad_norm
```

Outputs

```
in-place gradient step(s) on `policy`
stats dict: mean pg_loss, v_loss, entropy, approx_kl, clipfrac
```

(averaged over all minibatches x streams)

Procedure (optimize_batches)

```
n      <- streams[0].obs.shape[0]      # = n_envs * n_steps
mb_size <- max(1, n // num_minibatches)

# ---- actor-coefficient normalization (PPO-Lagrangian stability) ----
coeff_sum <- sum(actor_coeffs)
if coeff_sum > 0:
    actor_coeffs <- [c / coeff_sum for c in actor_coeffs]
# positive rescaling of the ascent direction: preserves relative past/current
# weighting (and the primal-dual fixed point), only bounds step magnitude.
# For the local phase ([1.0]) this is a no-op.

for epoch = 0 .. ppo_epochs-1:
    perm <- random permutation of 0..n-1
    for each minibatch idx in perm (step mb_size):
        total_loss <- 0
        for (coeff, batch) in zip(actor_coeffs, streams):
            dist, value <- policy.dist_value(batch.obs[idx], batch.task_id)
            logratio <- dist.log_prob(batch.actions[idx]) - batch.logprobs[idx]
            ratio <- exp(logratio)
            adv <- batch.advantages[idx]
            if normalize_advantage: adv <- (adv - mean)/(std + 1e-8)

            # clipped surrogate (maximize return => minimize negatives)
            pg_loss <- mean( max( -adv*ratio,
                                -adv*clamp(ratio, 1-clip_ratio, 1+clip_ratio) ) )
            v_loss <- 0.5 * mean( (value - batch.returns[idx])^2 ) # value MSE
            entropy <- mean( dist.entropy() )

        total_loss <- total_loss
                        + coeff * pg_loss          # weighted actor term
                        + vf_coef * v_loss         # standard critic term
                        - ent_coef* entropy        # entropy bonus

    # optional BC on the CURRENT (last) stream only:
    if bc and this is the last stream:
        total_loss <- total_loss
                        + bc_coef * mean( KL(pi_local || dist) )

    accumulate diagnostics: pg, v, entropy,
        approx_kl = mean(ratio - 1 - logratio),
        clipfrac  = fraction with |ratio-1| > clip_ratio
```

```
optimizer.zero_grad()
total_loss.backward()
clip_grad_norm_(policy.parameters(), max_grad_norm) # GLOBAL grad clip
optimizer.step()
```

```
return averaged {pg_loss, v_loss, entropy, approx_kl, clipfrac}
```

What is updated, and when

policy params : once per minibatch (ppo_epochs * num_minibatches steps total).
Only params with requires_grad move (whole net normally; heads only when the trunk is frozen for the head-only probe).

The critic and entropy terms are IDENTICAL for every stream (coeff 1 and ent_coef); ONLY the actor term carries the per-stream continual weight.

Key intermediate quantities

ratio	= exp(new_logp - old_logp)	: importance ratio
clipped surrogate		: PPO trust-region actor loss
v_loss	= 0.5*(V - returns)^2	: critic regression target
coeff (normalized)		: per-stream actor weight
approx_kl, clipfrac		: trust-region health diagnostics

Implicit assumptions

- All streams share ONE minibatch schedule and equal size $N = n_{\text{envs}} * n_{\text{steps}}$.
- advantages / returns are precomputed by the collector via GAE(gae_lambda).
- The shared critic is trained by the SAME value loss for every task on the shared trunk -- hence the coeff normalization to avoid starving it.
- Actions discrete (Categorical dist); log_prob / entropy per timestep.

Relation to the equations

clipped surrogate + value MSE + entropy standard PPO (optimizer only)
per-stream actor_coef = policy-gradient weight ... eqs 22, 28-29 (A) / 46 (B)
coeff normalization by sum engineering fix for the
unbounded dual coefficient $\mu^2 * \text{shortfall}$ (eq 30); preserves the fixed point
global grad-norm clip trust-region / stability